

Objective:

Design, build, and deploy a machine learning model for real-time credit card fraud detection. The goal is not just model accuracy but a robust, scalable, and maintainable MLOps pipeline on AWS or Azure

Dataset

You will use the Kaggle Credit Card Fraud Detection dataset

: <https://www.kaggle.com/mlg-ulb/creditcardfraud>

Tasks:

Build a binary classifier to predict the probability that a transaction is fraudulent.

The candidate must demonstrate proficiency in the following areas:

1. Environment & Reproducibility

- a. **Code Structure:** Create a well-organized Python project (e.g., using a ``src`` package structure).
- b. **Dependency Management:** Use ``poetry`` or ``pipenv`` to manage a virtual environment with pinned dependencies (`requirements.txt` or `pyproject.toml`).
- c. **Configuration:** Use a framework like ``Hydra``, ``OmegaConf``, or even a simple `config.yaml` to manage all parameters (paths, model hyperparameters, thresholds).

2. Data Handling & Versioning

- a. **Data Versioning:** Use DVC (Data Version Control) to version the dataset and any processed artefacts (train/test splits, preprocessed data). Track metrics alongside data commits.
- b. **Data Preprocessing:** Write scalable and reusable preprocessing code (e.g., ``scikit-learn`` Pipelines). Handle class imbalance appropriately (e.g., class weights, careful sampling techniques).

3. Model Development & Training

- a. **Training Script:** Create a script that trains at least two different models (e.g., ``XGBoost`` and ``RandomForest``).
- b. **Model Evaluation:** Go beyond accuracy. Use precision-recall curves, F1-score, AUC, and specifically focus on metrics meaningful for fraud detection (e.g., Precision@K, Recall@K). Document the choice of probability threshold.
- c. **Model Serialization:** Serialize the best model, its preprocessing pipeline, and metadata (e.g., using ``joblib`` or ``pickle``).

4. Versioning, Registry & CI/CD

- a. **Model Versioning:** Use MLflow to track all experiments, parameters, metrics, and artifacts. The final model must be registered in the **MLflow Model Registry**.
- b. **CI/CD** (GitHub Actions/GitLab CI):
 - Run unit tests and linting (``pytest``, ``pylint`/flake8``) on PRs.
 - Train the model and log to MLflow on a push to the ``main`` branch (or manually triggered).
 - Bonus: If metrics surpass a certain threshold, automatically transition the model to the ``Staging`` stage in the registry.

5. Deployment & Serving (Choose AWS or Azure)

- a. **Real-Time Inference Server:** Package the model into a Docker container.

- Use FastAPI to create a REST API endpoint (e.g., `/predict`).
- The API should accept a JSON payload with transaction data and return a JSON response with the fraud probability and a binary prediction based on a configurable threshold.
- Write a comprehensive `Dockerfile`.
- b. Cloud Deployment:** One of the following:
 - Deploy the Docker container. Use AWS ECR to store the Docker image.
 - Deploy the Docker container to Azure Container Instances (ACI).
 - Use Azure Container Registry (ACR) to store the Docker image.
- c. Infrastructure as Code (IaC):** Define the entire cloud infrastructure using Terraform or AWS CDK/Azure Bicep. This includes computing resources, networking, and permissions.

6. Monitoring & Observability

- a. **API Monitoring:** Instrument the FastAPI app with logging and metrics using `prometheus-client` and `grafana` (or cloud-native tools like AWS CloudWatch / Azure Monitor).
- b. **Model Monitoring:** Log all predictions (with a unique request ID) to a storage solution (e.g., S3 / Azure Blob Storage). This is crucial for future drift detection.
- c. **Health Checks:** Implement a `/health` endpoint for the inference service.

7. Documentation & Testing

- d. **API Documentation:** Auto-generate and deploy interactive API documentation using FastAPI's built-in Swagger UI (`/docs`).
- e. **Project README:** Write a comprehensive `README.md` with instructions on how to setup, run, train, test, and deploy the entire project.
- f. **Testing:** Include unit tests for data validation, model loading, and API logic.

Expected Deliverables:

- ✓ A public GitHub/GitLab repository with all code, configuration, and documentation.
- ✓ Clear commit history demonstrating a logical workflow.
- ✓ Terraform/CDK scripts that can be run to provision the required infrastructure.
- ✓ The trained model available in an MLflow Model Registry.
- ✓ A live, publicly accessible API endpoint (or instructions to deploy it easily) that can be queried for predictions.
- ✓ A screencast or a set of screenshots (2-3 mins) walking through the key functionalities:
 - Running the training pipeline.
 - Showing the MLflow experiment tracking UI.
 - Demonstrating a successful API call to the model.

Thanks for your time - **Team Wink !!**